Impact of Caching Strategies on Large-Scale Backend Application Performance

Yuvraj Prajapat

2210992573

Chitkara University Institute of Engineering and Technology ,

Chitkara university

Rajpura, Punjab,

India

Yuvraj2573.be22@chitkara.edu.in

Vansh Tomer

2210994857

Chitkara University Institute of Engineering and Technology ,

Chitkara university

Rajpura, Punjab,

India

Vansh4857.be22@chitkara.edu.in

ABSTRACT—

Modern backend development involves dealing with a number of performance issues. A common issue encountered with backend applications when they have a high volume of users and the same data repeated multiple times throughout the application is that the system becomes much less efficient; for example, the response time will be much greater. The research presented in this paper focuses on the different caching techniques and their effects on backend application performance. The study will also look at reducing latency and increasing throughput..

The methodology for this research involved creating a prototype of a backend using Node.js and Express.js with a large data set to test against in the experiments. To do so, I used three different methods of caching; in-memory, Redis based, and cache-aside, along with Autocannon to measure response time and throughput among others.

The researches have shown that systems are much faster with caching enabled. Systems which are not using any caching technique have the slowest response times and throughput when compared to systems that do use caching techniques. All of the various caching methods tested produced quantifiable improvements, with Memory-based caching producing the quickest response times. Redis-based caching had the greatest scaling capacity but did have slower overall response times when compared to other caching methods,.

These results clearly indicate that caching is the single most important factor for improving backend system performance and that developers must carefully choose which caching methods will be appropriate for their application.

Keywords:

Caching, Backend Performance, Redis, Node.js, Scalability, Latency Reduction Introduction

I. INTRODUCTION

The majority of digital services, including online marketplaces, social media services, and Web-based applications stored in the cloud, currently have their underlying systems built on large back-end applications. These large back-end applications are creating an ever-increasing number of requests from an ever-growing number of users, resulting in the need for speedy responses to those requests. Additionally, an increased number of users can create other challenges, such as high latency, increased load on the database, and slower processing times on back-end Servers.

One of the key contributors to application performance degradation is insufficient requests to the database [1][11]. Most applications have a tendency to repeatedly request the same data from the database, thus resulting in the database being overloaded and an increase in response time. This issue especially impacts heavily read applications due to the large number of requests made for the same data.

Caching is a commonly utilized method to resolve the previous problem [1][6][9]. Caching is defined as storing frequently requested data in a faster storage medium (i.e., RAM) so that subsequently requested data can be satisfied from cached data with no database interaction required.

Backend applications can use four primary caching strategies for caching using memory: in-memory caching can keep data in server memory; therefore, making access to that data faster than if stored in some other location, but it is not scalable.

Shared access to data between server nodes provides a distributed cache; using extensions like Redis enables server nodes to share their caches and increase overall application scalability [3][6].

The third cache strategy is also known as a cache-aside, where caches only are accessed when validation of data is required.

The intention of this research project is to explore how different caching strategies impact how effectively the backend application responds and interacts with users. To do this, we have built a prototype to analyse performance through different common caching strategies in various

scenarios. In addition to response times, each scenario will also look at throughput levels.

Modern backend systems need to serve a large number of concurrent users while at the same time keeping response times low and ensuring high availability. In these scenarios, it is essential to use resources efficiently to avoid server overload and provide a smooth experience for users. Different caching strategies offer different trade-offs in speed, scalability, memory usage and implementation complexity. Therefore, it is important to choose a suitable caching strategy to enhance the overall efficiency and scalability of backend applications. This research focuses on the practical implementation and real-time performance testing of these strategies to better understand their impact on modern backend systems.

The rest of the paper is organized as follows. Section II presents the literature review and discusses the related research work on caching strategies and backend optimization techniques. Section III describes the methodology, implementation details and experimental setup of the study. The performance results and findings obtained from the different caching strategies with graphical analysis are highlighted in Section IV. The observed results and a comparative analysis of the implemented caching approaches are summarized in Section V. Finally, Section VI concludes the paper and discusses possible future scope for further improvement and research.

II. LITERATURE REVIEW

Cache optimization has been widely studied in the academic literature due to its usefulness as a way of improving backend service performance, especially in situations where there is a lot of user activity and data is being accessed repeatedly [1][2][9][11]. Numerous scientific papers have noted how cache technologies can allow for the avoidance of continuously querying the database and speed up the entire process.

One of the simplest yet most powerful caching techniques is in-memory caching [1][9]. This technique revolves around storing repetitive data directly in RAM on the server. Accessing in-memory cache will be much quicker than performing normal data storage. Some of the more common technologies for creating in-memory caches include Redis and Memcached. Despite this fact, many scholarly sources suggest that there are issues when using this method due to limited memory capacity.

Recently, Redis-based distributed caches have become increasingly popular among researchers because they support high-volumes of data, since the data is distributed across many different nodes [2][3][6][9]. Thus, distributed caching not only improves scalability but also enhances fault tolerance [6][11]. Numerous academic papers have demonstrated that the use of distributed caching not only increases throughput rate but also reduces database activity during times of high concurrent access.

A common alternative caching strategy is cache aside, which loads data into cache from source to cache as required by the application. This approach allows for only necessary data to be cached in order to conserve memory usage. Research has shown that cache aside provides more efficient control of the cache and performs better than other caching strategies when dealing with read modifies or repetitive reads.

Additionally, prior research has demonstrated the impact of cache management techniques such as eviction strategies (referring to the method used to remove data from the cache) and cache hit rates on overall application performance. Efficient use of the cache will produce a high cache hit ratio, thus improving performance, however poor use of the cache will negatively impact performance. Well-known caching techniques that improve cache storage efficiencies include least-recently-used (LRU) and time-to-live (TTL) [1][9].

III. METHODOLOGY

A. Research Approach

A variety of experimental methodologies were used as the basis for evaluating the impact of different caching techniques on backend application performance. The central objective of this study is to determine the impact of caching on response time and throughput through the use of controlled testing. In order to conduct the experiment, a prototype version of a backend application was designed and implemented. This application was tested using a number of various caching methodologies as part of the evaluation process.

B. Performance Metrics

The backend application used for this experiment was developed using Node.js and Express.js [4]. The volume of data that was utilized for creating realistic examples makes it possible to have real-time results and performance metrics.

C. Evaluation Process

Each of the above mentioned cache solutions was evaluated using a test methodology as outlined below:

No Cache (Base configuration)

The No Cache configuration resulted in each request being resolved using the database directly without any caching mechanism in place resulting in longer processing and latency times.

In Memory Cache

An In Memory cache solution where the most frequently accessed data was stored in the server memory so when a request for the same piece of data was made, it didn't require the data to be re-processed (or computed).

Redis-Cache

Through the evaluation process, the use of a Redis cache solution, where the data is separated from the application and read from a separate source was reviewed [3].

Cache Aside

The `Cache Aside` strategy requires a check to see if the requested data is in cache, if it is not in cache, it would be retrieved from the database and saved into cache to facilitate future requests for that data.

D. Evaluation Metrics

The following evaluation metrics were used to gauge the performance of the system:

Latency (Response Time): Average period of time taken by a request to be processed [2][11].

Throughput: The number of requests processed per second.

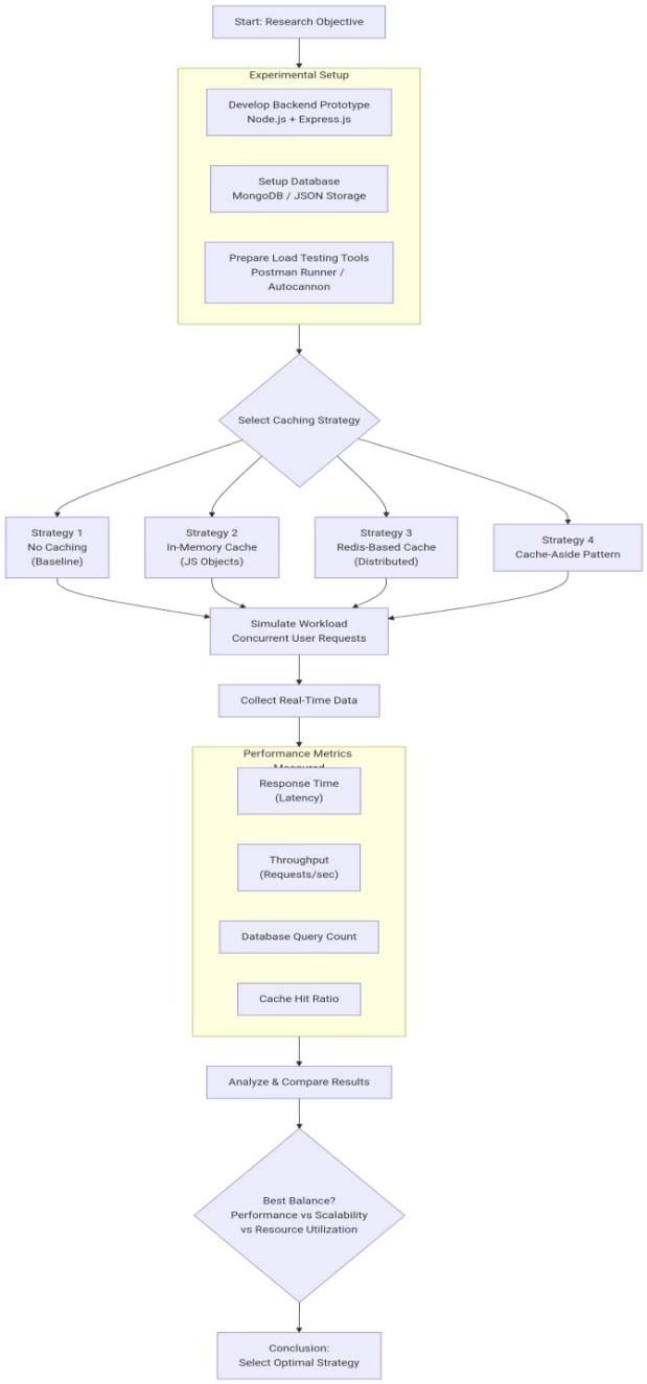
E. Evaluation Methods of Caching

All caching types were each evaluated independently in a similar environment (regarding endpoint & workloads). Multiple measurements were taken for each of the tested caching types to determine averages.

Most theoretical work is merely a reflection of assumed values, while this study is based on actual data collected from load testing [2][9].

The evaluation will focus on identifying which caching strategy provides the best balance between performance, scalability, and resource utilization.

Figure 1 shows the complete workflow of the proposed research methodology for analysis of caching strategies in backend applications. The process starts with backend setup and load-testing setup. Then there are four different caching approaches implemented: No Caching, In-Memory Cache, Redis-Based Cache and Cache-Aside Pattern. After that, we simulate concurrent user requests to gather performance metrics like response time, throughput, database query count, and cache hit ratio. Lastly, we analyse these metrics to identify the most efficient caching strategy for optimising backend performance.



` *Fig 1: Methodology flow*

IV. RESULTS / FINDINGS

By analysing the amount of time it takes for a cache system to respond and how many requests or transactions per second can be completed (transaction throughput) we can determine how well a cache system performs [2][9][11]. In all of these experiments each involved 50 user accessing the cache at the same time for 20 seconds so they can be compared with one another in a fair manner.

The average results obtained from multiple runs are summarized in the below table.

Strategy	Avg Latency (ms)	Avg Throughput(req/s)
No cache	123.84	567.49
Memory Cache	13.92	3543.83
Redis Cache	17.61	2843.60
Cache-Aside	12.48	3864.63

Table 1: performance comparison of caching strategies

Figure 2 shows that the use of caching methods leads to reductions in the amount of latency that occurs when accessing or retrieving data from the database[2][9]. Highest latencies occur with the no-cache method but the cache-aside method has the lowest response times of any of the caching methods

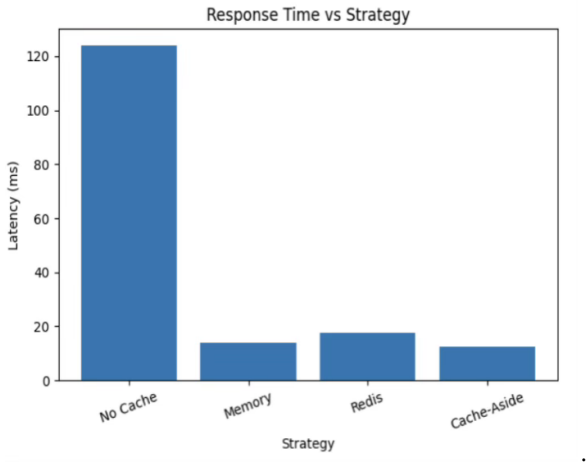


Figure 2 – Response Time vs strategy

Figure 3 shows power graphs found that there was a large amount of requests per second can be processed through caching methods. Cache aside techniques had the highest throughput, followed closely by in-memory caching methods, while Redis also performed very well comparatively to the base line [9].

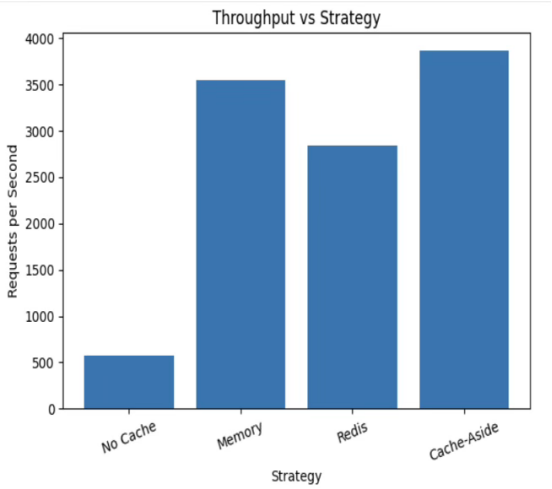


Figure 3 – Throughput Gauge for Cache Methods

Figure 4 shows use of various cache methods, latencies can be reduced by over 85% [2][9]. Cache aside techniques had the greatest amount of latency reduction percentage by way of the other caching methods tested.

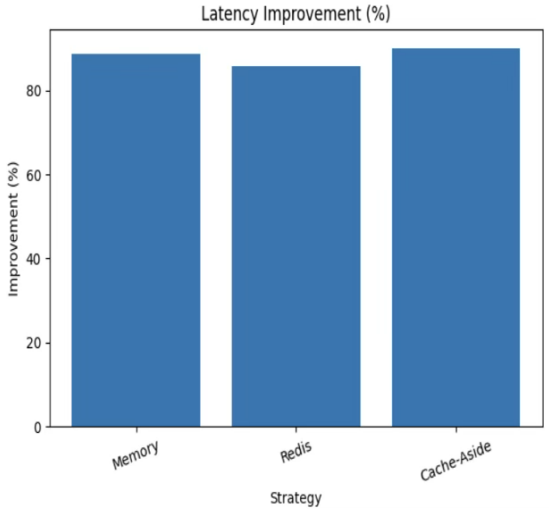


Figure 4 – Latency Reduction %

Figure 5 shows throughput graphs, ample capacity increase can be achieved using cache. Cache aside techniques will have the largest percentage increase in throughput with in-memory caching and Redis having moderate throughput rate increases.

The previously discussed graphs indicate that caching provides improved backend performance. Of the caching techniques tested, Cache aside has provided the greatest overall performance and has the quickest access speed (In-Memory), in addition to being highly scalable (Redis).

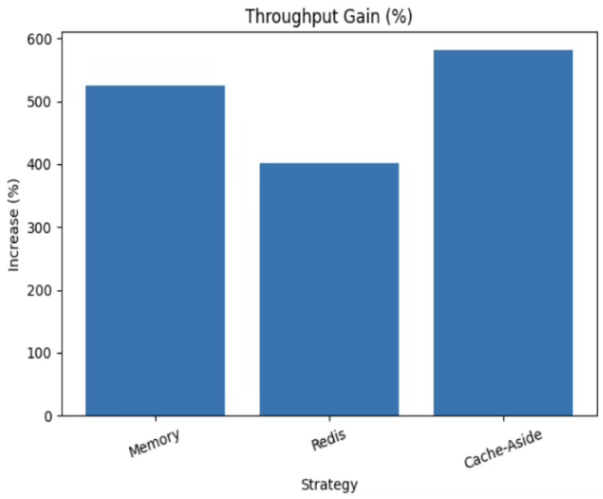


Figure 5 - % Increase in Throughput

V. DISCUSSION

The results indicate that caching techniques have greatly increased the performance of backend applications [1][2][9][11]. When caching techniques were implemented, response time was significantly lower and throughput was higher compared to no caching techniques being implemented.

Without any caching, maximum latency and minimum throughput were recorded due to the repeated computations performed over the same data and then accessing this data directly several times.

Therefore, it is evident that using just the backend application alone with repeated access to the same data is not beneficial.

Of all the different caching techniques that had been implemented, the in-memory caching technique produced the greatest level of performance improvements. With the data cached in memory, further computations were unnecessary and the response time improved dramatically. However, this technique does impose limits on memory size.

Redis-based caching provided a remarkably high improvement in performance, although it was still slightly slower than caching with simply storing the data entirely in memory; this will result in significantly larger data sets, however, while all being able to fit into the same amount of memory space [3][6]. Redis has experienced higher latencies than using only caching, mainly due to using Redis for networking and serialization purposes; however, it was able to process all of the queries with good throughput.

The cache-aside approach produced the greatest performance improvements in every single test [2][9]. That is, by only loading into the cache items that were required when they were used, the cache could be used at the highest level of performance with the lowest possible latencies and maximum possible throughput.

Another important perspective to take note of, is that the actual performance of any given process through caching, will depend upon how effectively caches are being utilized in terms of operational execution. Therefore, as one would expect, with an increased number of hits on a cache, one can expect to see an increase in the overall performance and vice versa [1][9].

In conclusion, based on the available evidence obtained during research, we can ascertain that caching is a very effective means of enhancing a backend system's performance [1][2][9][11]. The fastest way to carry out a process, is via either in memory caching solutions or via either Redis or cache aside solutions - as well as the scalability features they both provide.

Ultimately, it is abundantly clear that choosing which caching strategy best suits a particular application, requires taking into account multiple criteria, including, but not

limited to, the overall amount of data that an application generally processes; as well as how frequently that data is accessed by the application.

It is also important to point out that the true value of this research will come from the empirical evaluation of various caching strategies, in terms of their effectiveness, once implemented and tested in real world settings.

VI. CONCLUSION

By researching the impact of different caching methods on backend application performance through implementation-based analysis, it has become apparent that caching is important for the efficiency of the system [1][9].

The baseline system shows very slow performance because each request requires computation and retrieving data. All of the different caching methods were able to produce some type of performance improvement; however, In-Memory caching has had the most benefit because the data was accessed quickly from the local memory. Caching methods only work best when the amount of data being cached is small, and when scalability isn’t critical.

The use of a Redis-based caching strategy has been demonstrated to provide higher performance through improved scalability and successfully allowing for distributed system operation [3][6]. There was some overhead associated with network communications; however, there was also significant management of concurrency for request processing which resulted in achieving a high throughput.

Of all of the analysed caching strategies, the cache-aside caching strategy was determined to provide the best performance and efficiency [2][9]. This was accomplished due to caching on-demand and only caching data that was necessary and at the time that it was necessary, which resulted in much lower levels of overall overhead and latency than the other analysed caching strategies, and therefore the cash-aside caching strategy offered the highest throughput.

The analysis concluded that the caching approach selected should take into consideration a variety of factors, most importantly the need for scalability. The correct approach to caching can lead to improved performance and scalability within a system.

This study demonstrates the practical advantages of caching via real-time testing and offers insights for optimizing contemporary backend applications.

While the research validates that caching techniques can promote performance gain from the backend, there are still areas where additional study can improve the efficiency of caching strategies.

Some examples include increased use of larger and more realistic data sets. For this study we utilized a synthetic data set for experimentation. Future studies should test caching strategies on actual data sets. This will provide more usable results than what has been obtained in this study.

One approach to enhance our understanding of cache mechanisms is through the assessment of caching methodologies. Developing effective caching techniques such as Least Recently Used (LRU) and Time To Live (TTL) caching mechanisms will assist in optimizing memory utilization as well as increasing the effectiveness of caching systems [1][9].

Additionally, additional research can be performed regarding caching utilization within various application architectures that exist today. With the emergence of increasingly popular application architectures including microservices and distributed computing architectures, unique challenges exist when defining cache solutions within these architectures [6][11].

Finally, additional research can also explore using machine learning algorithms in the design phase of cache strategies. The implementation of machine learning in caching strategy development could potentially lead to more dynamic caching solutions based on the changes of workloads and provide greater cache efficiency over time.

Furthermore, future research may also exploit caching strategies designed for high levels of concurrent processes or different workload types.

In general, using more advanced methods and real-world situations to build on this research can help us better understand caching strategies and how they affect modern backend systems.

VII. REFERENCES

[1] Mohan, C., Caching Technologies for Web Applications, IBM Almaden Research Center, 2001.

Available: https://www.almaden.ibm.com/u/mohan/Caching_VLDB2001.pdf

[2] https://www.researchgate.net/publication/385916660_OPTIMIZING_APPLICATION_PERFORMANCE_A_STUDY_ON_THE_IMPACT_OF_CACHING_STRATEGIES_ON_LATENCY_REDUCTION

[3] Redis Documentation, “Redis Official Documentation”

Available: https://redis.io/documentation

[4] Node.js Documentation, “Node.js Official Docs”

Available: https://nodejs.org/en/docs

[5] Fielding, R., Architectural Styles and the Design of Network-based Software Architectures, 2000.

Available: https://www.ics.uci.edu/~fielding/pubs/dissertation/top.htm

[6] Amazon Web Services (AWS), “Caching Best Practices” Available: https://aws.amazon.com/caching

[7] Big picture performance analysis using Lighthouse Parade

[8] Nguyen, Testing Applications on the Web

[9] Piastou, Evaluating the Efficiency of Caching Strategies in Reducing Application Latency

[10] Crawford & Hussain, Comparison of Server Side Scripting Technologies

[11] Yang et al., Performance Optimization for Database-Backed Web Applications

Link with code used: https://github.com/vansh909/caching-performance-analysis.git